Manual técnico — ProyectoFinal

Frontend: React + Vite + TypeScript | Backend: NestJS + TypeScript | ORM: Prisma | DB: SQLite | Desktop: Electron

1. Alcance

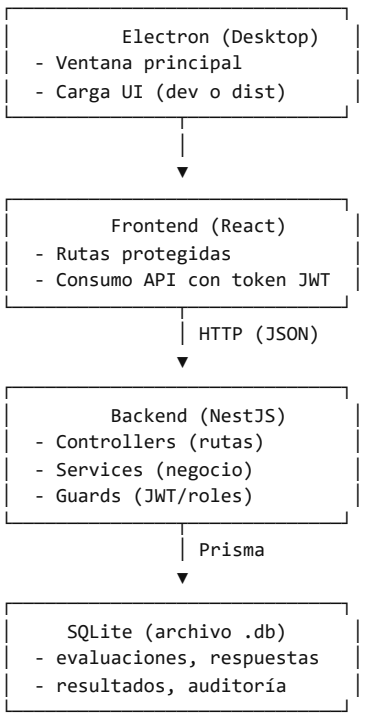
Aplicación para crear, ejecutar y analizar evaluaciones de usabilidad de múltiples interfaces, usando dimensiones ISO 9241-11 (Efectividad, Eficiencia, Satisfacción). Incluye gestión de usuarios, captura de respuestas, cálculo de resultados, auditoría y generación de PDF.

2. Tabla de contenido

3. Arquitectura

Diagrama lógico

text



4. Estructura del repositorio

Monorepo con tres paquetes:

- `backend`: API NestJS + Prisma
- `frontend`: UI React + Vite
- `desktop`: Electron (contenedor desktop)

5. Comandos de ejecución

Desarrollo (todo)

bash

```
npm install
npm run db:setup
npm run dev
```

Levanta backend + frontend + electron en paralelo.

Producción (desktop)

bash

```
npm run prod:desktop
```

Genera build del frontend y abre Electron usando el HTML estático.

6. Variables de entorno

Componente	Variable	Propósito	Ejemplo
Backend	DATABASE_URL	Ruta/URL de SQLite para Prisma	file:./prisma/dev.db
Backend	JWT_SECRET	Secreto para firmar JWT	(string segura)
Backend	PORT	Puerto del servidor	3000
Backend	NODE_ENV	Modo de ejecución	development
Seed	ADMIN_EMAIL	Email del admin inicial	admin@demo.com
Seed	ADMIN_PASSWORD	Password del admin inicial (mín. 8)	12345678
Seed	ADMIN_FULL_NAME	Nombre del admin inicial	Administrador
Frontend	VITE_API_URL	Base URL del backend	http://localhost:3000/api
Electron	ELECTRON_RENDERER_URL	Override de URL del renderer	http://localhost:5173

7. Backend (NestJS)

7.1 Bootstrap y validación global

ts

```ts
async function bootstrap() {
  const app = await NestFactory.create(AppModule, { bodyParser: false });
  app.use(json({ limit: '10mb' }));
  app.use(urlencoded({ extended: true, limit: '10mb' }));
  app.setGlobalPrefix('api');
  app.enableCors({ origin: true });
  app.useGlobalPipes(
    new ValidationPipe({
      whitelist: true,
      forbidNonWhitelisted: true,
      transform: true,
    }),
  );
  await app.listen(process.env.PORT ?? 3000);
}
```

7.2 Módulos (inyección de dependencias)

ts

```ts
@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true }),
    PrismaModule,
    AuthModule,
    UsersModule,
    EvaluationsModule,
  ],
})
export class AppModule {}
```

7.3 DTOs (validación de entrada)

LoginDto / RegisterDto

ts

```ts
export class LoginDto {
  @IsEmail()
  email!: string;

  @IsString()
  @MinLength(8)
  password!: string;
}

export class RegisterDto {
  @IsEmail()
  email!: string;

  @IsString()
  @MinLength(8)
  password!: string;

  @IsString()
  @MinLength(2)
  fullName!: string;
}
```

SubmitAnswersDto

ts

```ts
export class SubmitAnswersDto {
  @Type(() => Number)
  @IsInt()
  interfaceId!: number;

  @IsArray()
  @ArrayMinSize(1)
  @ValidateNested({ each: true })
  @Type(() => SubmitAnswerItemDto)
  answers!: SubmitAnswerItemDto[];
}
```

7.4 JWT (Autenticación)

ts

```ts
export type JwtUser = {
  userId: number;
  email: string;
  role: RoleName;
};

validate(payload: JwtPayload): JwtUser {
  return { userId: payload.sub, email: payload.email, role: payload.role };
}
```

7.5 Roles (Autorización)

ts

```ts
@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private readonly reflector: Reflector) {}

  canActivate(context: ExecutionContext): boolean {
    const requiredRoles = this.reflector.getAllAndOverride<RoleName[]>(
      ROLES_KEY,
      [context.getHandler(), context.getClass()],
    );
    if (!requiredRoles || requiredRoles.length === 0) return true;
    const request = context.switchToHttp().getRequest();
    const user = request.user;
    if (!user?.role) return false;
    return requiredRoles.includes(user.role);
  }
}
```

7.6 Controllers (rutas declaradas)

ts

```ts
@Controller('auth')
export class AuthController {
  @Post('register') register(@Body() dto: RegisterDto) { ... }
  @Post('login') login(@Body() dto: LoginDto) { ... }
}

@Controller('users')
export class UsersController {
  @UseGuards(JwtAuthGuard) @Get('me') me(...) { ... }
  @UseGuards(JwtAuthGuard, RolesGuard) @Roles(RoleName.ADMIN) @Get() listAll() { ... }
  @UseGuards(JwtAuthGuard, RolesGuard) @Roles(RoleName.ADMIN) @Patch(':id/role') updateRole(...) { ... }
}

@UseGuards(JwtAuthGuard)
@Controller('evaluations')
export class EvaluationsController {
  @Post() create(...) { ... }
  @Get() listMine(...) { ... }
  @Get(':id') getById(...) { ... }
  @Patch(':id/scoring-weights') updateScoringWeights(...) { ... }
  @Post(':id/answers') submitAnswers(...) { ... }
  @Post(':id/compute-results') computeResults(...) { ... }
}
```

8. Prisma + Base de datos

Prisma define el modelo de datos y genera el cliente de acceso a la base de datos.

8.1 PrismaService (conexión y lifecycle)

ts

```ts
@Injectable()
export class PrismaService extends PrismaClient implements OnModuleInit, OnModuleDestroy {
  async onModuleInit() {
    await this.$connect();
  }
  async onModuleDestroy() {
    await this.$disconnect();
  }
}
```

8.2 Enums (estado y dimensiones)

prisma

```prisma
enum EvaluationStatus {
  DRAFT
  IN_PROGRESS
  COMPLETED
}

enum UsabilityDimension {
  EFFECTIVENESS
  EFFICIENCY
  SATISFACTION
}
```

8.3 Campos relevantes en Evaluation

prisma

```prisma
model Evaluation {
  id           Int @id @default(autoincrement())
  title        String
  systemName   String
  userType     UserType
  usageContext String

  scoringWeightEffectiveness Float @default(1)
  scoringWeightEfficiency    Float @default(1)
  scoringWeightSatisfaction  Float @default(1)
  status       EvaluationStatus @default(DRAFT)

  result       Result?
  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt
}
```

9. API (rutas)

Prefijo global: `/api`

Módulo	Ruta	Método	Descripción	Auth
Health	/api/health	GET	Chequeo de vida	No
Auth	/api/auth/register	POST	Registro (crea usuario EVALUATOR por defecto)	No
Auth	/api/auth/login	POST	Login, devuelve JWT	No
Users	/api/users/me	GET	Perfil del usuario autenticado	JWT
Users	/api/users	GET	Listar usuarios	JWT + ADMIN
Users	/api/users/:id/role	PATCH	Cambiar rol de usuario	JWT + ADMIN

9.1 Evaluations (núcleo)

Ruta	Método	Descripción	Permiso
/api/evaluations	POST	Crear evaluación	JWT
/api/evaluations	GET	Listar evaluaciones visibles	JWT
/api/evaluations/:id	GET	Detalle de evaluación	JWT
/api/evaluations/:id	DELETE	Eliminar evaluación	JWT + canManage
/api/evaluations/:id/scoring-weights	PATCH	Actualizar pesos de scoring	JWT + canManage
/api/evaluations/:id/compute-results	POST	Calcular resultado final	JWT + canManage
/api/evaluations/:id/report	GET	Reporte (JSON)	JWT
/api/evaluations/:id/report.pdf	GET	Reporte (PDF)	JWT
/api/evaluations/:id/results-breakdown	GET	Breakdown final	JWT
/api/evaluations/:id/interface-breakdown	GET	Resultados por interfaz (dinámico)	JWT
/api/evaluations/:id/completeness	GET	Checklist/Completitud	JWT
/api/evaluations/:id/interfaces	POST	Agregar interfaz	JWT + canManage
/api/evaluations/interfaces/:interfaceId	PATCH	Editar interfaz	JWT + canManage
/api/evaluations/interfaces/:interfaceId	DELETE	Eliminar interfaz	JWT + canManage
/api/evaluations/interfaces/:interfaceId/questions	POST	Agregar pregunta	JWT + canManage
/api/evaluations/questions/:questionId	PATCH	Editar pregunta	JWT + canManage
/api/evaluations/questions/:questionId	DELETE	Eliminar pregunta	JWT + canManage
/api/evaluations/interfaces/:interfaceId/tasks	POST	Agregar tarea	JWT + canManage
/api/evaluations/tasks/:taskId	PATCH	Editar tarea	JWT + canManage
/api/evaluations/tasks/:taskId	DELETE	Eliminar tarea	JWT + canManage
/api/evaluations/:id/answers	POST	Guardar respuestas	JWT + canSubmit
/api/evaluations/:id/answers?interfaceId=...	GET	Ver mis respuestas por interfaz	JWT
/api/evaluations/:id/selection	GET	Ver mi interfaz seleccionada	JWT
/api/evaluations/:id/selection	POST	Seleccionar interfaz	JWT + canSubmit
/api/evaluations/:id/task-attempts	POST	Registrar intento de tarea	JWT + canSubmit

Ruta	Método	Descripción	Permiso
/api/evaluations/:id/manual-user-stories	GET	Listar historias manuales	JWT
/api/evaluations/:id/manual-user-stories	POST	Crear historia manual	JWT + canManage
/api/evaluations/:id/manual-user-stories/:storyId	PATCH	Editar historia manual	JWT + canManage
/api/evaluations/:id/manual-user-stories/:storyId	DELETE	Eliminar historia manual	JWT + canManage
/api/evaluations/:id/evaluators	GET	Listar evaluadores asignados	JWT + canManage
/api/evaluations/:id/evaluators	POST	Asignar evaluador (por email)	JWT + canManage
/api/evaluations/:id/evaluators/:evaluatorId	DELETE	Quitar evaluador	JWT + canManage
/api/evaluations/:id/sessions/start	POST	Iniciar sesión de evaluación	JWT + canSubmit
/api/evaluations/:id/sessions/end	POST	Finalizar sesión de evaluación	JWT + canSubmit
/api/evaluations/:id/audit	GET	Ver auditoría	JWT + canManage

9.2 Lógica clave de resultados (extractos)

Cálculo de score global (ponderado)

ts

```
const overallScore = clamp01(
  wSum > 0
    ? (effectivenessScore * wEff +
        efficiencyScore * wEfi +
        satisfactionScore * wSat) / wSum
    : (effectivenessScore + efficiencyScore +
satisfactionScore) / 3,
);
```

Invalida resultado al cambiar pesos

ts

```
if (changed && existing.result) {
  await tx.result.delete({ where: { evaluationId }
});
  await tx.evaluation.update({
    where: { id: evaluationId },
    data: { status: EvaluationStatus.IN_PROGRESS },
  });
}
```

10. Módulo Evaluations (núcleo)

10.1 Normalización de respuestas (para puntajes 0..1)

ts

```
function clamp01(value: number) {
  if (Number.isNaN(value)) return 0;
  return Math.max(0, Math.min(1, value));
}

function normalizeAnswer({ questionType, dimension, valueNumber, valueLikert, valueBoolean }): number | null {
  if (questionType === QuestionType.BOOLEAN) return valueBoolean ? 1 : 0;
  if (questionType === QuestionType.LIKERT_1_5) return clamp01(((valueLikert ?? 0) - 1) / 4);
  if (questionType === QuestionType.NUMBER) {
    const n = valueNumber ?? 0;
    if (dimension === UsabilityDimension.EFFICIENCY) return clamp01(1 - n / 100);
    return clamp01(n / 100);
  }
  return null;
}
```

10.2 Permisos (ver / gestionar / registrar datos)

ts

```
private async assertCanViewEvaluation(user: JwtUser, evaluationId: number) {
  if (user.role === RoleName.ADMIN) return;

  const evaluation = await this.prisma.evaluation.findUnique({
    where: { id: evaluationId },
    select: {
      createdById: true,
      status: true,
      evaluators: {
        where: { evaluatorId: user.userId },
        select: { id: true },
      },
    },
  });

  if (!evaluation) throw new NotFoundException('Evaluación no encontrada');
  if (evaluation.createdById === user.userId) return;
  if (evaluation.evaluators.length > 0) {
    if (evaluation.status === EvaluationStatus.DRAFT) throw new ForbiddenException();
    return;
  }
  throw new ForbiddenException();

private async assertCanSubmitEvaluationData(user: JwtUser, evaluationId: number) {
private async assertCanSubmitEvaluationData(user: JwtUser, evaluationId: number) {
  if (user.role === RoleName.ADMIN) {
    throw new ForbiddenException(
      'El administrador no puede registrar sesiones, intentos ni respuestas',
    );
  }

    where: { id: evaluationId },
    select: { status: true, evaluators: { where: { evaluatorId: user.userId }, select: { id: true } } },
    select: {
      status: true,
      evaluators: {
        where: { evaluatorId: user.userId },
        select: { id: true },
      },
    },
  if (!evaluation) throw new NotFoundException('Evaluación no encontrada');

  if (evaluation.evaluators.length > 0) {
    if (evaluation.status === EvaluationStatus.DRAFT) throw new ForbiddenException('La evaluación aún está en
borrador...');
    if (evaluation.status === EvaluationStatus.DRAFT) {
      throw new ForbiddenException(
        'La evaluación aún está en borrador. Espera a que el administrador la habilite.',
```

```
      );
    }
  }
  throw new ForbiddenException();
}
```

10.3 Guardado de respuestas (no editables + obligatorias)

ts

```
const existingAnswersCount = await this.prisma.answer.count({
  where: { evaluationId, evaluatorId: user.userId, interfaceId: dto.interfaceId },
});
if (existingAnswersCount > 0) {
  throw new ConflictException('Las respuestas para esta interfaz ya fueron guardadas y no se pueden modificar.');
}

const missingRequired = intf.questions
  .filter((q) => q.isRequired)
  .filter((q) => !dto.answers.some((a) => a.questionId === q.id));
if (missingRequired.length > 0) {
  throw new BadRequestException('Faltan respuestas obligatorias');
}

await this.prisma.answer.createMany({ data: rows });
await this.prisma.evaluation.update({ where: { id: evaluationId }, data: { status: EvaluationStatus.IN_PROGRESS } });
```

10.4 Asignación de evaluador (habilita evaluación si estaba en borrador)

ts

```
const created = await this.prisma.evaluationEvaluator.upsert({
  where: { evaluationId_evaluatorId: { evaluationId, evaluatorId: evaluator.id } },
  update: {},
  create: { evaluationId, evaluatorId: evaluator.id, assignedById: user.userId },
});

await this.prisma.evaluation.updateMany({
  where: { id: evaluationId, status: EvaluationStatus.DRAFT },
  data: { status: EvaluationStatus.IN_PROGRESS },
});
```

10.5 Sesiones e intentos (métricas operativas)

ts

```
await this.prisma.evaluationSession.updateMany({
  where: { evaluationId, evaluatorId: user.userId, status: SessionStatus.IN_PROGRESS },
  data: { status: SessionStatus.COMPLETED, endedAt: new Date() },
});
const created = await this.prisma.evaluationSession.create({
  data: { evaluationId, evaluatorId: user.userId, status: SessionStatus.IN_PROGRESS },
});

await this.prisma.taskAttempt.upsert({
  where: { evaluationId_taskId_evaluatorId: { evaluationId, taskId: dto.taskId, evaluatorId: user.userId } },
  update: { completed: dto.completed, errorsCount, timeSec: dto.timeSec ?? null, stepsCount: dto.stepsCount ?? null,
notes: dto.notes ?? null },
  create: { evaluationId, taskId: dto.taskId, evaluatorId: user.userId, completed: dto.completed, errorsCount, timeSec:
dto.timeSec ?? null, stepsCount: dto.stepsCount ?? null, notes: dto.notes ?? null },
});
```

10.6 Checklist de completitud (antes de calcular)

ts

```
const hasAnyData = totalAnswers > 0;
const hasStructure = interfaces.some((i) => i.questions.length > 0);
const hasPending = evaluators.some((e) => e.selection === null || e.missingRequiredAnswersCount > 0);
const readyToCompute = hasStructure && hasAnyData;
```

```
return {
  summary: { hasStructure, hasAnyData, totalAnswers, evaluatorsCount: evaluators.length, readyToCompute, hasPending },
  evaluators,
};
```

10.7 Auditoría (trazabilidad)

ts

```
await this.prisma.auditLog.create({
  data: {
    evaluationId,
    actorId: params.user.userId,
    action: params.action,
    entityType: params.entityType,
    entityId,
    data: (params.data ?? null) as never,
  },
});
```

10.8 Cálculo de métricas (ponderación por dimensión)

ts

```
const [questions, answers] = await Promise.all([
  this.prisma.question.findMany({
    where: { interface: { evaluationId } },
    select: { id: true, dimension: true, type: true, weight: true },
  }),
  this.prisma.answer.findMany({
    where: { evaluationId, ...(evaluatorId ? { evaluatorId } : {}) },
    select: { questionId: true, valueNumber: true, valueLikert: true, valueBoolean: true },
  }),
]);

const questionById = new Map(questions.map((q) => [q.id, q]));
const agg = new Map<UsabilityDimension, { sum: number; weightSum: number }>([
  [UsabilityDimension.EFFECTIVENESS, { sum: 0, weightSum: 0 }],
  [UsabilityDimension.EFFICIENCY, { sum: 0, weightSum: 0 }],
  [UsabilityDimension.SATISFACTION, { sum: 0, weightSum: 0 }],
]);

let countedAnswers = 0;
for (const a of answers) {
  const q = questionById.get(a.questionId);
  if (!q) continue;
  const v = normalizeAnswer({
    questionType: q.type,
    dimension: q.dimension,
    valueNumber: a.valueNumber,
    valueLikert: a.valueLikert,
    valueBoolean: a.valueBoolean,
  });
  if (v === null) continue;
  countedAnswers += 1;
  const slot = agg.get(q.dimension);
  if (!slot) continue;
  slot.sum += v * q.weight;
  slot.weightSum += q.weight;
}
```

10.9 Resumen estadístico por evaluador (promedio/mediana)

ts

```
const avg = (xs: number[]) =>
  xs.length ? xs.reduce((a, b) => a + b, 0) / xs.length : 0;

const median = (xs: number[]) => {
  if (xs.length === 0) return 0;
  const sorted = [...xs].sort((a, b) => a - b);
```

```
  const mid = Math.floor(sorted.length / 2);
  if (sorted.length % 2 === 1) return sorted[mid] ?? 0;
  return ((sorted[mid - 1] ?? 0) + (sorted[mid] ?? 0)) / 2;
};

return {
  evaluatorsSummary: {
    count: breakdown.length,
    average: { overallScore: avg(scores.overall) },
    median: { overallScore: median(scores.overall) },
  },
};
```

11. Frontend (React)

11.1 Router compatible con Electron

ts

```ts
const Router = window.location.protocol === 'file:' ? HashRouter : BrowserRouter;
```

11.2 Rutas protegidas (auth/rol)

tsx

```tsx
function Protected({ children }) {
  const { token } = useAuth();
  if (!token) return <Navigate to="/login" replace />;
  return <>{children}</>;
}

function AdminOnly({ children }) {
  const { token, user } = useAuth();
  if (!token) return <Navigate to="/login" replace />;
  if (user?.role?.name !== 'ADMIN') return <Navigate to="/" replace />;
  return <>{children}</>;
}
```

11.3 Cliente HTTP genérico

ts

```ts
export async function apiRequest<T>(path: string, options: RequestInit & { token?: string } = {}): Promise<T> {
  const baseUrl = getApiBaseUrl();
  const res = await fetch(`${baseUrl}${path}`, {
    ...options,
    headers: {
      'Content-Type': 'application/json',
      ...(options.token ? { Authorization: `Bearer ${options.token}` } : {}),
      ...(options.headers ?? {}),
    },
  });

  if (!res.ok) {
    const text = await res.text();
    throw { status: res.status, message: text || res.statusText };
  }
  if (res.status === 204) return undefined as T;
  return (await res.json()) as T;
}
```

11.4 Persistencia de autenticación

ts

```ts
useEffect(() => {
  if (token && user) {
    localStorage.setItem(storageKey, JSON.stringify({ token, user }));
  } else {
    localStorage.removeItem(storageKey);
  }
}, [token, user]);
```

11.5 Flujo en la pantalla de Evaluación (pasos y validaciones)

Gating de pasos (estructura antes de ejecutar)

tsx

```tsx
const allSteps = [
  { id: 'userStories', title: '1) Historias de
```

Guardar respuestas (obligatorias)

tsx

```tsx
const missingRequired = selectedInterface.questions
  .filter((q) => q.isRequired)
```

```
usuario' },
  { id: 'interfaces', title: '2) Interfaces' },
  { id: 'structure', title: '3) Estructura' },
  { id: 'run', title: '4) Evaluación' },
  { id: 'results', title: '5) Resultados' },
];

const stepAvailability = useMemo(() => {
  const hasStories = manualUserStories.length > 0;
  const hasInterfaces = interfaces.length > 0;
  const hasQuestions = hasInterfaces &&
interfaces.some((i) => i.questions.length > 0);
  return {
    run: { enabled: hasStories && hasInterfaces &&
hasQuestions },
    results: { enabled: hasStories && hasInterfaces
&& hasQuestions },
    // ...
  };
}, [manualUserStories, interfaces]);
```

```
  .filter((q) => {
    const v = selectedAnswers[q.id];
    if (q.type === 'TEXT') return !String(v ??
'').trim();
    if (q.type === 'BOOLEAN') return v !== 'true' &&
v !== 'false' && v !== true && v !== false;
    if (q.type === 'LIKERT_1_5') { const n =
Number(v); return !Number.isFinite(n) || n < 1 || n >
5; }
    const n = Number(v); return !Number.isFinite(n);
  });
if (missingRequired.length > 0) {
  setActionError('Completa todas las preguntas
obligatorias antes de guardar.');
  return;
}
```

11.6 Calcular resultados (usa checklist y confirma si hay pendientes)

tsx

```
if (!completeness) {
  const loaded = await onLoadCompleteness();
  if (!loaded) return;
  if (!loaded.summary.hasStructure || !loaded.summary.hasAnyData) return;
}
if (completeness?.summary.hasPending) {
  const ok = window.confirm('Hay evaluadores sin interfaz seleccionada o preguntas obligatorias sin responder...');
  if (!ok) return;
}
await apiRequest(`/evaluations/${evaluationId}/compute-results`, { method: 'POST', token });
```

12. Desktop (Electron)

12.1 Script de arranque en desarrollo (espera servicios)

json

```
"dev": "wait-on --timeout 120000 http://localhost:5173 http://localhost:3000/api/health && electron ."
```

12.2 Carga de UI (dev vs dist)

js

```
function getRendererUrl() {
  if (process.env.ELECTRON_RENDERER_URL) return process.env.ELECTRON_RENDERER_URL;
  const lifecycleEvent = process.env.npm_lifecycle_event;
  if (!app.isPackaged && lifecycleEvent === 'dev') return 'http://localhost:5173';
  if (fs.existsSync(distIndexPath)) return pathToFileURL(distIndexPath).toString();
  return 'http://localhost:5173';
}
```

12.3 Configuración de seguridad

js

```
webPreferences: {
  contextIsolation: true,
  nodeIntegration: false,
  sandbox: true,
  preload: path.join(__dirname, 'preload.cjs'),
}
```